

msa_zria

msa_zria combines:

- MSA-style LLM parsing and model synthesis
- ZRIA symbolic reasoning
- Pyro-based probabilistic execution

What MSA And ZRIA Mean Here

We implement Model Synthesis Architecture (MSA) from arxiv paper 2507.12547v1. The code defines **MSA** as a staged reasoning workflow built around three typed steps:

- **parse** into a **ParseTarget**
- **code** into a **CodeTarget**
- **evaluate** into an **EvaluationTarget**

That workflow is executed by the **ReasoningPipeline**, which runs:

1. query parsing
2. reasoning-program synthesis
3. controlled Pyro execution
4. final evaluation

So in this repository, **MSA** means a multi-stage LLM-assisted reasoning loop with explicit intermediate contracts rather than a single free-form model response.

The code defines **ZRIA** as a separate decision backend layer behind the **BaseZRIAAdapter** and **BaseZRIABackend** interfaces.

ZRIA can run through three backend types:

- **rules**: explicit rule matching over query text, parsed state, and KG scope
- **learned**: a local trained classifier with confidence-based fallback
- **remote**: an external service client with fallback

Each backend returns an **EvaluationTarget**, and the trace model records which backend was configured, which backend actually produced the answer, and whether fallback fired.

So in this repository, **ZRIA** means the structured decision layer that can answer a case directly through rules, a learned local model, or a remote service, independently of the Pyro path.

What msa_zria Can Solve

msa_zria is a fit for problems where a plain LLM answer is not enough, and where you need some combination of:

- structured fact extraction
- explicit reasoning over rules or probabilities

- auditable intermediate steps
- domain adaptation on proprietary data

In practice, that means `msa_zria` is most useful for systems that must turn messy inputs into structured state, reason over that state, and then return an answer or action recommendation.

Good Problem Types

- Customer support diagnosis where product facts, known issues, and resolution policies matter
- Operational decision support where uncertainty must be modeled explicitly
- Compliance or policy checking where the system must explain why an action is allowed or blocked
- Incident triage where free-text reports must be converted into structured cases before routing
- Knowledge-grounded assistants that need both retrieval from a graph and a reasoning layer on top

Example Use Cases

1. Technical Support Triage

Input:

```
"After last night's storm, my router keeps dropping out every 10 minutes."
```

`msa_zria` flow:

- parse the report into device, issue, trigger, and severity
- retrieve router and outage facts from the knowledge graph
- synthesize a probabilistic or rule-based model of likely causes
- produce a recommendation such as `check line stability`, `power cycle router`, or `escalate to field technician`

Why `msa_zria` fits:

- the input is unstructured
- the answer depends on domain facts
- the output benefits from explicit reasoning instead of pure text generation

2. Equipment Failure Prediction

Input:

```
"Pump A overheats when run above 80% load for more than 15 minutes."
```

`msa_zria` flow:

- extract variables and thresholds
- map them to equipment facts and prior failure modes
- generate a Pyro model for failure likelihood under different loads

- return a risk estimate and mitigation recommendation

Why `msa_zria` fits:

- uncertainty matters
- you want a model-derived answer, not just a plausible sentence

3. Policy and Procedure Guidance

Input:

```
"Can this refund be approved if the item was opened after 45 days?"
```

`msa_zria` flow:

- parse the request into policy fields
- retrieve refund rules and exception clauses
- run symbolic or probabilistic reasoning over the policy state
- return `approve`, `deny`, or `manual review`, plus explanation

Why `msa_zria` fits:

- policy reasoning is structured
- the output should be inspectable and defensible

From Data to Production

The intended delivery pipeline for `msa_zria` is:

1. Gather Raw Domain Data

Collect the inputs your target system actually uses:

- support tickets
- chat transcripts
- troubleshooting playbooks
- policy documents
- tabular incident logs
- knowledge graph triples
- outcome labels such as `resolved`, `failed`, `escalated`, or `refund_approved`

At this stage, the goal is not model training yet. The goal is to capture the real domain states, rules, and outcomes.

2. Normalize and Structure the Facts

Convert the raw data into reusable assets:

- triples for entities, relations, conditions, and actions
- natural language summaries of those triples

- supervised examples for parse, code generation, and evaluation tasks

Typical outputs here:

- `triples.jsonl`
- `nl.jsonl`
- `hybrid.jsonl`
- evaluation sets with expected answers or outcomes

3. Define the Reasoning Contract

Before training, define exactly what the system must produce:

- parse schema: for example `device`, `issue`, `cause`, `severity`
- code or reasoning schema: for example a Pyro program, a rule trace, or a structured decision plan
- final output schema: for example `answer`, `confidence`, `recommended_action`, `justification`

This is the most important design step. If the schemas are vague, the training data and evaluation will drift.

4. Build Training Data for the Three Main Tasks

`msa_zria` is designed around three task families:

- parsing
- reasoning/code synthesis
- evaluation

For each real scenario, generate examples such as:

- parse the user report into structured fields
- generate a reasoning program or rule trace from the parsed facts
- judge whether a proposed answer solves the problem

This gives the model a role in the MSA loop instead of training it as a generic chatbot.

5. Fine-Tune Gemma 4 12B

Use the Gemma 4 12B QLoRA path in this repo to adapt the base model to your domain:

- choose the GPU profile for xpu or cuda
- train on the parse/code/evaluate dataset
- save the adapter or merged model
- run small regression checks on held-out cases

The fine-tuned model should be better at producing the exact structured outputs your reasoning stack needs.

6. Add the External Reasoning Layer

Once the model can reliably parse and synthesize reasoning steps, wire it into:

- ZRIA for symbolic reasoning

- Pyro for probabilistic reasoning
- KG retrieval for domain facts

The production system should not rely on the LLM alone when the task needs exact logic or uncertainty handling.

7. Evaluate the Full Pipeline

Test the end-to-end system on realistic cases:

- does parsing recover the right state?
- does retrieval supply the right supporting facts?
- does the Pyro or ZRIA stage produce the right decision?
- does the final explanation match the computed result?

This is where you compare:

- Gemma-only
- ZRIA-only
- Pyro-only
- hybrid `msa_zria`

8. Deploy as a Modular Service

A production `msa_zria` deployment should be modular rather than monolithic:

- ingestion service for raw requests
- retrieval service for triples and domain facts
- Gemma inference service for parse/code/evaluate stages
- reasoning service for Pyro or ZRIA execution
- API layer for the final user-facing answer

That structure makes it easier to inspect failures, replace components, and version models independently.

9. Log, Review, and Retrain

After deployment, capture:

- user queries
- parsed outputs
- retrieved facts
- generated reasoning artifacts
- final decisions
- human overrides or corrections

Those records become the next round of supervised data, which is how the system should improve over time.

Model Target

This repository targets:

- Base model: [google/gemma-4-12B](#)
- Processor template source: [google/gemma-4-12B-it](#)
- Training style: QLoRA with Hugging Face [transformers](#), [peft](#), and [trl](#)

Why this model:

- It is positioned for reasoning, coding, and agentic workflows.
- It supports long context and modern chat/system-role formatting.
- It aligns the repository with the Gemma 4 model family and instruction processor.

Current Capabilities

The repository includes the core pieces needed to exercise the full [msa_zria](#) loop:

- typed experiment configuration
- WWKG branch/workspace context in the experiment config
- canonical dataset and evaluation contracts
- customer-support ingestion into canonical JSONL records
- a [trl.SFTTrainer](#) training entrypoint
- Gemma 4 12B model, processor, quantization, and LoRA setup
- DSPy-facing parse/code/evaluate module prompts
- a controlled Pyro execution harness with syntax validation and timeouts
- a trainable local ZRIA backend plus rules and remote backend support
- a narrow reasoning pipeline: parse -> code -> Pyro -> evaluate
- an ablation runner with JSON report output
- integration tests for runtime, pipeline, backend matching, and API routes
- sample source cases, training records, eval records, and ablation cases

Repository Layout

```
msa_zria/
├── README.md
├── configs/
│   ├── gemma4_12b.yaml
│   ├── gemma4_12b_a770.yaml
│   └── gemma4_12b_v100.yaml
├── examples/
│   ├── ablation_cases.jsonl
│   ├── customer_support_cases.jsonl
│   ├── customer_support_records_eval.jsonl
│   ├── customer_support_records_train.jsonl
│   ├── zria_examples_eval.jsonl
│   ├── zria_examples_train.jsonl
│   └── zria_rules.json
├── pyproject.toml
├── src/
│   └── msa_zria/
│       ├── __init__.py
│       ├── ablation.py
│       └── config.py
```

```
├─ data.py
├─ ingest.py
├─ pyro_runtime.py
├─ reasoning_pipeline.py
├─ train.py
├─ training.py
├─ zria.py
├─ zria_adapter.py
└─ zria_backend.py
```

Design Choices

1. Use `AutoModelForMultimodalLM`

Gemma 4 12B uses the multimodal Gemma 4 stack even when the current workload is text-only:

```
from transformers import AutoModelForMultimodalLM, AutoProcessor
```

2. Use `AutoProcessor`, not just `AutoTokenizer`

Gemma 4 uses an official processor and chat template. This is the right place to keep prompt formatting consistent across training and inference.

3. Keep the first implementation text-first

The project goal mentions customer support parsing, code synthesis, and evaluation. Those can all start as text tasks even though Gemma 4 12B also supports image and audio inputs.

4. Keep the implementation narrow

This repo includes a narrow working slice, not a full production platform. The goal is to make each stage concrete and testable without pretending the domain logic, retrieval layer, or ZRIA implementation are complete.

Quick Start

Create an environment and install the package:

```
cd msa_zria
python -m venv .venv
source .venv/bin/activate
pip install -e .
```

Ingest example customer-support source cases into canonical training records:

```
python -m msa_zria.ingest \
  --input examples/customer_support_cases.jsonl \
  --output examples/customer_support_records_train.jsonl \
  --input-mode hybrid \
  --workspace urn:wwkg:workspace:example \
  --branch support-hotfix
```

Load the example config and initialize the Gemma 4 12B training objects:

```
from msa_zria.config import load_experiment_config
from msa_zria.training import (
    build_lora_config,
    build_sft_config,
    load_model_and_processor,
)

config = load_experiment_config("configs/gemma4_12b.yaml")
model, processor = load_model_and_processor(config.model)
peft_config = build_lora_config(config.training)
sft_config = build_sft_config(config.training)
```

Build a training example from triples:

```
from msa_zria.data import Triple, TrainingExample

example = TrainingExample(
    instruction="Extract the device, issue, and recommended action.",
    triples=[
        Triple(subject="Printer_X", predicate="issue", object="PaperJam"),
        Triple(subject="Printer_X", predicate="resolution",
object="ClearPaperPath"),
    ],

    response='{"device":"Printer_X","issue":"PaperJam","resolution":"ClearPaper
Path"}',
)

messages = example.to_messages(input_mode="hybrid")
```

Run supervised fine-tuning with the canonical JSONL records:

```
python -m msa_zria.train --config configs/gemma4_12b.yaml
```

Run the baseline ablation harness and write a JSON report:

```
python -m msa_zria.ablation --config configs/gemma4_12b.yaml
```

Train the learned local ZRIA backend:

```
python -m msa_zria.zria train \  
  --train examples/zria_examples_train.jsonl \  
  --eval examples/zria_examples_eval.jsonl \  
  --output outputs/zria_learned
```

Compare the learned backend against the rules backend:

```
python -m msa_zria.zria compare \  
  --model outputs/zria_learned \  
  --rules examples/zria_rules.json \  
  --input examples/zria_examples_eval.jsonl
```

Run the integration tests:

```
PYTHONPATH=src python -m unittest discover -s tests -v
```

Run the repo CI-equivalent locally:

```
./scripts/run_ci.sh
```

Run the live WWKG checks once WWKG is available:

```
WWKG_BASE_URL=http://127.0.0.1:4242 \  
WWKG_WORKSPACE=urn:wwkg:workspace:example \  
WWKG_BRANCH=main \  
./scripts/run_live_wwkg_checks.sh
```

Config Model

The example config separates model and training concerns:

```
experiment_name: msa-zria-gemma4-12b  
seed: 42  
model:  
  base_model_id: google/gemma-4-12B
```

```
processor_id: google/gemma-4-12B-it
load_in_4bit: true
training:
  output_dir: outputs/gemma4_12b
  learning_rate: 2.0e-4
  num_train_epochs: 3
  per_device_train_batch_size: 1
  gradient_accumulation_steps: 16
```

The config also includes:

- `kg.backend`
- `kg.workspace`
- `kg.branch`
- `kg.commit`
- `kg.as_of`
- `kg.graph_iri`
- `kg.sparql_query`
- `zria.backend`
- `zria.rules_path`
- `zria.learned_model_path`
- `zria.confidence_threshold`
- `zria.remote_url`
- `zria.fallback_to_rules`
- `audit.enabled`
- `audit.output_path`
- `audit.wwkg_enabled`
- `audit.promotion_output_path`
- `data.train_path`
- `data.eval_path`
- `ablation.cases_path`
- `ablation.output_path`

KG Context

`msa_zria` is branch/workspace-aware through a shared KG context model.

That context can be carried through:

- experiment config via `kg`:
- dataset production via `/produce_dataset`
- ingestion via `CustomerSupportCase.kg_scope` or CLI flags
- runtime requests via `kg_scope`
- ablation reports via `kg_metadata`

The default example configs point at WWKG and set:

```
kg:
  backend: wwkg
  base_url: http://127.0.0.1:4242
  workspace: urn:wwkg:workspace:example
  branch: main
```

To use a local-file graph path instead, set:

```
kg:
  backend: oxigraph
  graph_path: data/domain_graph.nq
  graph_format: nquads
```

Audit and Lineage

`msa_zria` has a first-class audit trail with local JSONL output and an optional WWKG mirror.

The audit surface covers:

- dataset lineage: source case id, workspace and branch, ingestion timestamp, input file hash, output record ids
- model lineage: experiment config hash, dataset version, model artifact path and hash, backend type, fallback setting, confidence threshold
- decision lineage: query, parsed state, backend used, whether fallback fired, final `EvaluationTarget`, operator override
- control events: Pyro timeout, disallowed AST or import rejection, remote backend failure, low-confidence learned fallback to rules
- validation evidence: ablation report ids, per-backend scores, learned-versus-rules comparison reports
- promotion events: approval of a branch into a production workspace

Example ingestion with audit enabled:

```
python -m msa_zria.ingest \
  --input examples/customer_support_cases.jsonl \
  --output examples/customer_support_records_train.jsonl \
  --input-mode hybrid \
  --workspace urn:wwkg:workspace:example \
  --branch support-hotfix \
  --audit-path outputs/audit/audit.jsonl
```

Example branch-promotion record:

```
python -m msa_zria.audit promote \
  --output outputs/audit/promotions.jsonl \
  --source-workspace urn:wwkg:workspace:staging \
```

```
--source-branch support-hotfix \  
--production-workspace urn:wwkg:workspace:prod \  
--approver compliance@example.com \  
--evidence-report-id ablation-report-20260622
```

Contract Validation

`msa_zria` includes a contract-validation CLI for the parse -> code -> Pyro -> evaluate slice.

Use the baseline heuristic modules:

```
python -m msa_zria.validate_contracts \  
--input examples/customer_support_cases.jsonl \  
--module-source baseline \  
--output outputs/contract_validation.json
```

Use the default Gemma-backed modules when the runtime is installed and `LM_PATH` is set:

```
LM_PATH=outputs/gemma4_12b \  
python -m msa_zria.validate_contracts \  
--input examples/customer_support_cases.jsonl \  
--module-source default \  
--output outputs/contract_validation_gemma.json
```

Dataset Formatting

`src/msa_zria/data.py` supports three input modes:

- `triples`
- `text`
- `hybrid`

This maps to three supported dataset presentations:

- structured triples only
- natural language only
- both together

Each `TrainingExample` renders to OpenAI-style chat messages:

- `system`
- `user`
- `assistant`

This keeps the dataset compatible with modern supervised fine-tuning pipelines and Gemma 4 chat templating.

Canonical Dataset Shape

The repository has a concrete canonical training record shape. Each JSONL row should map to a `DatasetRecord` with:

```
{
  "example_id": "parse-router-overheat-001",
  "task": "parse",
  "input_mode": "hybrid",
  "messages": [
    {
      "role": "system",
      "content": "You are a customer support reasoning assistant. Use the provided facts and return only valid JSON that matches the requested schema."
    },
    {
      "role": "user",
      "content": "Extract the device, issue, cause, and severity.\n\nFacts:\nRouter123 | hasIssue | Overheating\n\nContext:\nRouter123 has issue Overheating."
    },
    {
      "role": "assistant",
      "content": "{\n  \"task\": \"parse\",\n  \"device\": \"Router123\",\n  \"issue\": \"Overheating\",\n  \"cause\": null,\n  \"severity\": \"high\"\n}"
    }
  ],
  "target": {
    "task": "parse",
    "device": "Router123",
    "issue": "Overheating",
    "cause": null,
    "severity": "high"
  },
  "metadata": {
    "split": "train",
    "domain": "customer_support"
  }
}
```

The allowed target contracts are:

- `parse`
 - `device`, `issue`, `cause`, `severity`
- `code`
 - `language`, `framework`, `entrypoint`, `query_variable`, `required_statements`, `program`
- `evaluate`
 - `verdict`, `resolved`, `should_escalate`, `explanation`

This is the dataset shape the rest of the repo should treat as canonical.

Source Case Ingestion Shape

The ingestion script accepts `CustomerSupportCase` JSONL rows. A source case contains:

- `case_id`
- `customer_message`
- `candidate_answer`
- `triples`
- `context`
- `kg_scope`
- `parse_target`
- `code_target`
- `evaluation_target`

See [customer_support_cases.jsonl](#) for concrete examples.

Evaluation Contract

Evaluation is defined as a deterministic contract instead of a vague natural-language judgment.

`src/msa_zria/data.py` provides:

- `EvaluationCase`
- `EvaluationResult`
- `evaluate_case(...)`

Scoring rules:

- `parse`
 - exact normalized match on populated expected fields: `device`, `issue`, `cause`, `severity`
- `code`
 - exact match on `language`, `framework`, `entrypoint`, `query_variable`
 - substring presence checks for every item in `required_statements`
 - `program` must be non-empty
- `evaluate`
 - exact match on `verdict`, `resolved`, `should_escalate`
 - `explanation` is preserved for inspection but is not used for pass/fail

This gives the project a stable pass/fail contract for offline benchmarks and regression tests.

Training Entry Point

The training path is implemented in [train.py](#).

It does the following:

- reads `ExperimentConfig`
- loads canonical JSONL records
- renders each `messages` list through the Gemma chat template

- builds the model, processor, LoRA config, and SFT config
- launches `trl.SFTTrainer`
- saves the adapter and processor to the configured output directory

This is the supported path for Gemma 4 12B fine-tuning in this repository.

Narrow End-to-End Slice

The minimal end-to-end `msa_zria` flow is implemented in `reasoning_pipeline.py`:

1. Parse a query into `ParseTarget`
2. Generate a `CodeTarget`
3. Execute the generated program through `pyro_runtime.py`
4. Evaluate the result into `EvaluationTarget`

The Pyro runner expects generated code to define `run_inference()`. It validates the generated AST, restricts imports and builtins, executes in a separate worker process, and enforces a timeout.

ZRIA Adapter

The ZRIA integration point is defined in `zria_adapter.py`:

- `BaseZRIAAdapter`
- `ConfiguredZRIAAdapter`
- `RuleBasedZRIAAdapter`

The rules backend is loaded from `zria_rules.json` through `zria_backend.py`. The trainable local backend is implemented in `zria.py`, and a remote client backend also lives in `zria_backend.py`.

Supported backend types:

- `rules`
- `learned`
- `remote`

Example learned-backend config:

```
zria:
  backend: learned
  learned_model_path: outputs/zria_learned
  confidence_threshold: 0.6
  fallback_to_rules: true
```

Example remote-backend config:

```
zria:
  backend: remote
```

```
remote_url: http://zria.local/predict
fallback_to_rules: true
```

Rules remain the fallback backend for both `learned` and `remote` when the learned model is low-confidence or the remote service is unavailable.

Ablation Runner

The ablation harness is implemented in `ablation.py`.

It currently compares:

- `pyro_only`
- `zria_only`
- `hybrid`

The runner emits:

- per-case details
- per-mode accuracy
- per-mode average score

Sample ablation inputs live in `ablation_cases.jsonl`.

The CLI entrypoint reads `ablation.cases_path` and `ablation.output_path` from `ExperimentConfig`:

```
python -m msa_zria.ablation --config configs/gemma4_12b.yaml
```

The runner uses a narrow baseline parse/code/evaluate pipeline plus the rule-backed ZRIA adapter, so the report path is runnable before a real DSPy-backed Gemma inference stack is wired in. It also carries the configured WWKG scope into the report metadata so branch-level comparisons can be tracked explicitly.

Gemma 4 12B Loading Notes

The model-loading path does the following:

- selects the accelerator explicitly: `cuda`, `xpu`, or `auto`
- uses `bfloat16` on newer CUDA GPUs when available, `float16` on V100 and Intel Arc
- configures BitsAndBytes 4-bit NF4 quantization
- loads `AutoModelForMultimodalLM`
- loads the Gemma 4 instruction processor
- optionally calls `prepare_model_for_kbit_training`

The LoRA config intentionally does **not** hardcode target modules. Current Gemma 4 QLoRA guidance allows PEFT's Gemma defaults to scope the LM layers.

16GB GPU Profiles

The fine-tuning stage has separate accelerator-aware profiles for both of your planned cards:

- [configs/gemma4_12b_a770.yaml](#)
- [configs/gemma4_12b_v100.yaml](#)

Practical defaults:

- Intel Arc A770 16GB: `training.accelerator: xpu, float16, adamw_torch`
- NVIDIA V100 16GB: `training.accelerator: cuda, float16, paged_adamw_8bit`

The code keeps 4-bit QLoRA enabled for both GPU paths. If `training.accelerator` is `auto`, CUDA is preferred when available, then XPU, then CPU. For Intel GPU runs, make sure the environment exposes `torch.xpu`; upstream Intel XPU support in bitsandbytes exists, but is currently documented as less mature than the CUDA path.

Remaining Gaps

The large missing pieces are runtime quality and domain depth rather than missing scaffolding:

- real DSPy runtime validation against installed dependencies
- higher-quality learned ZRIA datasets and model tuning beyond the small local classifier scaffold
- a real deployed remote ZRIA service, not just the client and fallback path
- stronger process isolation for Pyro than a restricted worker process
- richer retrieval and KG integration
- larger real-world datasets and benchmark suites
- hardware validation of actual Gemma training runs

Notes

- The project targets Gemma 4 12B assumptions.
- It is intentionally a scaffold, not a finished reasoning system.
- The next useful milestone is stronger runtime validation and production hardening.